Reviewer Pack — Minimal Rank of Geometric Quantization over XOR-AND Tree Wid…

Entry 99e1deaf3bec · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-26 17:33:06 UTC

Minimal Rank of Geometric Quantization over XOR-AND Tree Width

Entry ID: `99e1deaf3bec` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-26 17:33:06 UTC

1. Conjecture

Field A (mathematical branch): Quantum Information Theory (Geometric Quantization) **Field B** (complexity object): Complexity Theory: XOR-AND Tree Width

Statement:

> ['For a given n-variable Boolean function f, the minimal rank of its corresponding quantum state ρ_f in geometric quantization is bounded by a function of the XOR-AND tree width T(f):', 'rk(ρ_f) ≤ α * log(T(f)) for some constant α.', 'Conversely, there exists a family of Boolean functions with XOR-AND tree width T(f) such that rk(ρ_f) ≥ β * T(f) for any quantum state ρ_f associated with f, where β is another constant.']

Rationale (proposer's reasoning):

> ['Geometric quantization offers a framework to represent classical information in terms of quantum states. By associating quantum states with Boolean functions, we can explore the interplay between quantum and computational complexity.', 'The XOR-AND tree width measures the computational difficulty of evaluating a Boolean function, making it a natural complexity measure to relate to geometric quantization.', 'If this conjecture holds, it would imply a connection between quantum information theory and classical computation that has not been previously explored.']

Taxonomy category: `Geometric_Quantization_XOR_AND_Tree_Width` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `d71f45b302a19973`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if at least 80% of generated n-variable Boolean functions have their quantum state rank rk(ρ_f) within ±3 standard deviations of α * log(T(f)) and falsified if any seed produces a metric value |rk(ρ_f) - α * log(T(f))| > 3 or |rk(ρ_f) - β * T(f)| > 3.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `quantum information theory AND geometric quantization AND XOR-AND tree width` - `minimal rank of quantum states AND geometric quantization AND complexity theory` - `Boolean functions AND XOR-AND tree width AND quantum state rank`

Top relevant hits considered: - [http://arxiv.org/abs/quant-ph/0305192v1] Photon engineering for quantum information processing - [http://arxiv.org/abs/1108.5136v1] Scalable Architecture for Quantum Information Processing with Atoms in Optical Micro-Structures - [http://arxiv.org/abs/2004.10708v2] Geometric distinguishability measures limit quantum channel estimation and discrimination - [http://arxiv.org/abs/2512.10504v2] Tianyan: Cloud services with quantum advantage - [http://arxiv.org/abs/1406.1964v2] Geometric global quantum discord of two-qubit X states - [http://arxiv.org/abs/2501.11119v1] Classical (ontological) dual states in quantum theory and the minimal group representation Hilbert space - [http://arxiv.org/abs/2008.00266v1] On parity decision trees for Fourier-sparse Boolean functions - [http://arxiv.org/abs/1102.1242v2] A refinement of Stone duality to skew Boolean algebras

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.3s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def generate_random_boolean_function(n):
    if n == 1:
        return random.choice(['0', '1'])
    else:
        op = random.choice(['AND', 'OR', 'XOR'])
        subformulas = [generate_random_boolean_function(random.randint(1, n)) for _ in range(rando
        return [op] + subformulas

def compute_xor_and_tree_width(formula):
    if isinstance(formula[0], list):
        left_width = compute_xor_and_tree_width(formula[1])
        right_width = compute_xor_and_tree_width(formula[2])
        return max(left_width, right_width) + 1
    else:
        return 1
```

```python
def geometric_quantization_rank(formula):
    if isinstance(formula[0], list):
        left_rank = geometric_quantization_rank(formula[1])
        right_rank = geometric_quantization_rank(formula[2])
        return max(left_rank, right_rank) + 1
    else:
        return 1

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n_values = [5, 10, 15, 20, 30, 40]
    results = []

    for n in n_values:
        formula = generate_random_boolean_function(n)
        xor_and_width = compute_xor_and_tree_width(formula)
        rank = geometric_quantization_rank(formula)

        if xor_and_width == 0 or rank == 0:
            continue

        alpha = Fraction(1, 2)  # Example constant
        beta = Fraction(1, 4)   # Example constant

        expected_min_rank = alpha * math.log(xor_and_width)
        expected_max_rank = beta * xor_and_width

        results.append({
            "n": n,
            "xor_and_width": xor_and_width,
            "rank": rank,
            "expected_min_rank": expected_min_rank,
            "expected_max_rank": expected_max_rank
        })

    metric_value = sum(result["rank"] for result in results) / len(results)
    instances_tested = len(results)
    conjecture_holds = all(expected_min_rank <= result["rank"] <= expected_max_rank for result in
    counterexample = "" if conjecture_holds else "rank out of bounds"

    return {
        "metric_name": "geometric_quantization_rank",
        "metric_value": metric_value,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2, 3, 5, 7, 11, 13, 17, 19, 23, 2

    results = []
```

```python
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{'seed': {seed}, **result}}")
        results.append(result)

    mean_value = sum(result["metric_value"] for result in results) / len(results)
    std_value = math.sqrt(sum((result["metric_value"] - mean_value) ** 2 for result in results) /
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif any(abs(result["metric_value"] - result["expected_min_rank"]) > 3 or abs(result["metric_v
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample='rank out of bounds' first_failing_seed={first_fa
    else:
        print("RESULT: INCONCLUSIVE insufficient data")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0862022f.py", line 90, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0862022f.py", line 49, in run_trial
    formula = generate_random_boolean_function(n)
              ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0862022f.py", line 23, in generate_rand
   subformulas = [generate_random_boolean_function(random.randint(1, n)) for _ in range(random.randi
                  ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0862022f.py", line 23, in generate_rand
   subformulas = [generate_random_boolean_function(random.randint(1, n)) for _ in range(random.randi
                  ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0862022f.py", line 23, in generate_rand
   subformulas = [generate_random_boolean_function(random.randint(1, n)) for _ in range(random.randi
                  ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  [Previous line repeated 993 more times]
  File "/usr/lib/python3.12/random.py", line 336, in randint
    return self.randrange(a, b+1)
           ^^^^^^^^^^^^^^^^^^^^^^
  File "/usr/lib/python3.12/random.py", line 318, in randrange
    return istart + self._randbelow(width)
                    ^^^^^^^^^^^^^^^^^^^^^^
RecursionError: maximum recursion depth exceeded
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test code crashed before producing data, which means it did not complete its execution to verify the conjecture. | next: Re-run the test with increased recursion limits and ensure that it completes without crashing.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	17422
2	preregistration	ollama_remote	glm4:latest	0	0	5912
3	novelty	ollama_remote	glm4:latest	0	0	4645
4	novelty	ollama_remote	glm4:latest	0	0	6474
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15269
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11596
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12503
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10909
9	judge	ollama_remote	glm4:latest	0	0	12517

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 97246 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/99e1deaf3bec.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/99e1deaf3bec.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/99e1deaf3bec.tar.gz` (if generated)